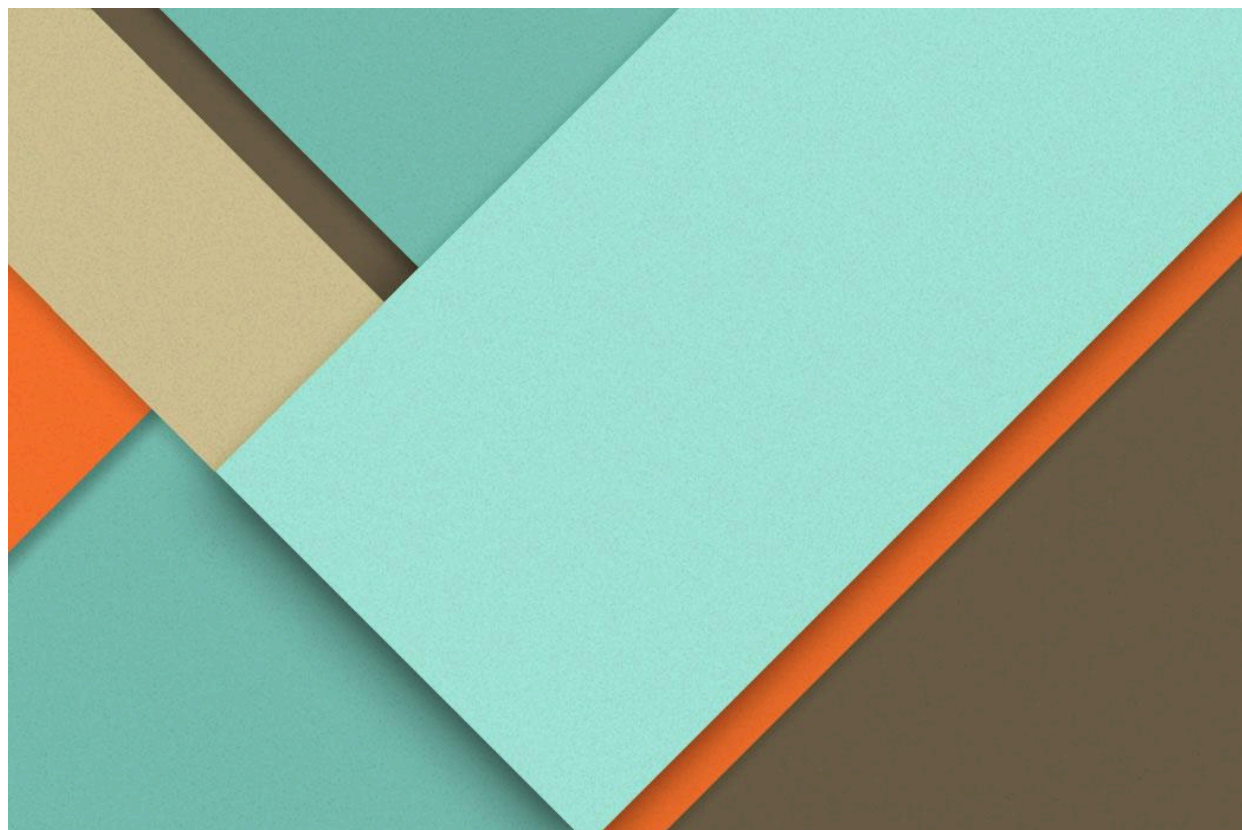




Boutique Tech 221 API



Groupe 5

Membre

Zeynab Ba

Gorgui Merena

Cheikh Ibrahima Thiam

Vue d'ensemble

Le projet **Boutiques TECH 221** est une API RESTful développée avec **Node.js**, **Express** et **Prisma** pour gérer les opérations liées aux boutiques et à leurs employés.

Elle permet d'effectuer des opérations CRUD complètes sur les **magasins**, les **employés**, les **produits**, et les **ventes**, tout en garantissant l'intégrité des données et la cohérence des relations entre les entités.

La documentation interactive est intégrée via **Swagger UI**.

Objectifs

1. Fournir une **API robuste et évolutive** pour la gestion des boutiques.
2. Implémenter **des validations strictes** pour éviter les incohérences et les duplications de données.
3. Permettre la gestion des employés, y compris le **soft delete** et la restauration.
4. Assurer une **documentation complète** et accessible via Swagger.
5. Maintenir une **architecture modulaire** et propre (controllers, services, repositories).

Caractéristiques

- CRUD complet pour les magasins , employés , produits et ventes
- **Soft delete** afin de ne jamais supprimer définitivement des données critiques
- **Validations robustes** (champs obligatoires, formats de téléphone, existence du magasin, unicité)
- **Gestion des erreurs standardisée** avec codes HTTP appropriés
- **Documentation Swagger** interactive et structurée
- **Architecture modulaire** avec séparation nette entre :
 - Controllers (logique route)
 - Services (logique métier)
 - Repositories (accès à la base via Prisma)
 - Middlewares (erreurs et validations)

Grandes étapes

I. Initialisation du projet

Cloner le projet

```
git clone https://github.com/digitalmaith/boutiques_TECH_221.git
```

```
cd boutiques_TECH_221
```

Création du projet Node.js et installation des dépendances (Express, Prisma, Zod, Swagger, etc.)

Configuration de la base PostgreSQL et du fichier `.env`.

II. Définition du schéma Prisma

- Modèles : `Magasin`, `Employe`, `Produit`, `Vente`
- Relations entre les entités et contraintes d'unicité.

III. Implémentation de la logique métier

- Services pour créer, mettre à jour, supprimer et restaurer les entités.
- Soft delete pour les employés avec champ `deletedAt`.

IV. Création des routes et des contrôleurs

- Routes REST pour toutes les entités.
- Validation des entrées via Zod et middleware `validate.js`.

V. Gestion des erreurs

- Centralisation des erreurs avec `httpError` et middleware `errorHandler`.

VI. Documentation Swagger

- Définition des schémas, routes et exemples.

- Intégration dans `app.js` pour accès à `/api-docs`.

VII. Tests et validation

- Vérification de la cohérence des données et du comportement des endpoints.
- Test des cas d'erreurs et duplication.

VIII. Projet

2. Architecture du projet

```
gestion-produit/
├─ package.json
├─ .env
├─ prisma/
│  ├─ schema.prisma           # Modèles et relations
│  └─ migrations/             # Migrations Prisma
├─ src/
│  ├─ app.js                  # Initialisation Express et Swagger
│  ├─ server.js               # Serveur
│  ├─ controllers/            # Logique des routes
│  │  ├─ magasin.controller.js
│  │  └─ employe.controller.js
│  ├─ services/               # Logique métier
│  │  ├─ magasin.service.js
│  │  └─ employe.service.js
│  ├─ repositories/           # Accès aux données via Prisma
│  │  └─ magasin.repo.js
```

```

| |   └─ employe.repo.js
| └─ routes/                # Définition des routes
|   └─ magasin.routes.js
|   └─ employe.routes.js
| └─ swagger/
|   └─ magasin.swagger.js  # documentation du swagger
|   └─ employe.swagger.js
└─ middlewares/            # Gestion erreurs et validation
    └─ errorHandler.js
    └─ validate.js
└─ utils/                  # Helpers
    └─ httpError.js
    └─ response.js
└─ swagger.js              # Configuration Swagger

```

4. Configuration Prisma

prisma/schema.prisma : modèles principaux

Magasin

```

// Modèle Magasin
model Magasin {
  id      Int      @id @default(autoincrement())
  nom     String
  adresse String
  ville   String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  employees Employee[]

  @@unique([nom, ville])
}

```

Employe

```

model Employe {
  id      Int      @id @default(autoincrement())
  prenom  String
  nom     String
  poste   String
  telephone String
  magasinId Int
  magasin Magasin @relation(fields: [magasinId], references: [id])
  deletedAt DateTime? // Soft delete
  ventes  Vente[]
  @@index([magasinId])
}

```

Produit & Vente

```

model Produit { ... }
model Vente { ... }

```

5. Routes API et Documentation Swagger

5.1 Magasins

- **GET /api/magasins** → Liste tous les magasins
- **GET /api/magasins/:id** → Récupérer un magasin par ID
- **POST /api/magasins** → Créer un magasin
- **PUT /api/magasins/:id** → Mettre à jour un magasin
- **DELETE /api/magasins/:id** → Supprimer un magasin

Exemple de Swagger :

```

/Magasins:
post:
  summary: Créer un magasin
  requestBody:
    required: true
  content:

```

```

    application/json:
      schema:
        $ref: '#/components/schemas/MagasinCreatePayload'
  responses:
    201:
      description: Magasin créé
    409:
      description: Magasin déjà existant

```

5.2 Employés

- **GET /api/employees** → Récupérer tous les employés actifs
- **GET /api/employees/deleted** → Récupérer tous les employés supprimés (soft delete)
- **GET /api/employees/:id** → Récupérer un employé par ID
- **POST /api/employees** → Créer un employé
- **PUT /api/employees/:id** → Mettre à jour un employé
- **DELETE /api/employees/:id** → Soft delete
- **PATCH /api/employees/:id/restore** → Restaurer un employé supprimé

Exemple Swagger POST :

```

/api/employees:
  post:
    summary: Créer un employé
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/EmployeeCreatePayload'
          example:
            prenom: Ali
            nom: Ndiaye
            poste: CAISSIER
            telephone: "771234567"

```

magasinId: 1
responses:
201:
description: Employé créé
400:
description: Données invalides ou magasin inexistant

5.3 Validation et contraintes

- Les champs obligatoires sont vérifiés via **Zod / middleware validate.js**.
 - **MagasinId** doit exister pour créer un employé.
 - **Soft delete** : `deletedAt` est utilisé pour archiver sans supprimer physiquement.
 - Un employé ne peut pas être dupliqué dans le même magasin avec **prénom + nom + téléphone**.
-

5.4 Gestion des erreurs

- Utilisation d'un helper `httpError(status, message)`
- Middleware `errorHandler.js` pour capturer les erreurs et renvoyer :

```
{  
  "message": "id invalide",  
  "status": 400,  
  "stack": "..."  
}
```

- 400 → Bad Request
- 404 → Not Found
- 409 → Conflit (duplicate entry)
- 500 → Internal Server Error

6. exemple de services

[magasin.service.js](#)

```
import magasinRepository from "../repositories/magasin.repo.js";
import httpError from "../utils/httpError.js";

class MagasinService {
  async updateMagasin(id, payload) {
    const magasin = await magasinRepository.findById(id);

    if (!magasin) {
      return null;
    }

    const data = {};
    if (payload.nom !== undefined) data.nom = payload.nom;
    if (payload.adresse !== undefined) data.adresse = payload.adresse;
    if (payload.ville !== undefined) data.ville = payload.ville;

    return magasinRepository.updateById(id, data);
  }

  async deleteMagasin(id) {
    const magasin = await magasinRepository.findById(id);

    if (!magasin) {
      return null;
    }

    await magasinRepository.deleteById(id);
    return true;
  }

  async createMagasin(payload) {
    // Vérification minimale
  }
}
```

```
if(!payload.nom || !payload.adresse || !payload.ville){
  throw httpError(400, "nom, adresse et ville sont obligatoires")
}

// On essaie de créer le magasin via le repository

try {
  const newMagasin = await magasinRepository.create(payload);
  return newMagasin
} catch (error) {
  // Gestion de l'erreur unique constraint (Prisma P2002)
  if (error.code === "P2002") {
    throw httpError(409 , "Ce magasin existe déjà dans cette ville");
  }
  throw error;
}
}

// récupère tous les magasins
async getAllMagasins(){
  return magasinRepository.findAll();
}

// récupère un magasin par ID

async getMagasinById(id) {
  if (!id) {
    return null;
  }
  return magasinRepository.findById(id);
}
}

export default new MagasinService();
```

[employe.service.js](#)

```
import employeeRepository from "../repositories/employee.repo.js";
import magasinRepository from "../repositories/magasin.repo.js";
import httpError from "../utils/httpError.js";

class EmployeeService {
  // Créer un employé
  async createEmployee(payload) {
    const magasin = await magasinRepository.findById(payload.magasinId);
    if (!magasin) throw new Error("Le magasin spécifié n'existe pas");

    // Vérifier doublon avant création
    const existing = await employeeRepository.findOne({
      prenom: payload.prenom,
      nom: payload.nom,
      telephone: payload.telephone,
      magasinId: payload.magasinId,
    });
    if (existing) throw new Error("Employé déjà existant");

    return employeeRepository.create(payload);
  }

  // Mettre à jour un employé
  async updateEmployee(id, data) {
    if (data.magasinId) {
      const magasin = await magasinRepository.findById(data.magasinId);
      if (!magasin) throw new Error("Le magasin spécifié n'existe pas");
    }

    return employeeRepository.update(id, data);
  }

  // Récupérer tous les employés non supprimés
  async getAll() {
    return employeeRepository.findAll();
  }
}
```

```
// Récupérer un employé par id
async getById(id) {
  return employeeRepository.findById(id);
}

// Soft delete d'un employé
async deleteEmployee(id) {
  return employeeRepository.softDelete(id);
}

// restaurer un employé
async restoreEmployee(id) {
  const employee = await employeeRepository.findByIdIncludeDeleted(id); //
  Vérifie que l'employé existe
  if (!employee) {
    throw httpError(404, "Employé introuvable");
  }
  return employeeRepository.restore(id);
}

async getDeletedEmployees() {
  return employeeRepository.findDeleted();
}
}

export default new EmployeeService();
```

8. exemple de Controller

magasin

```
import magasinService from "../services/magasin.service.js";
import httpError from "../utils/httpError.js";
import response from "../utils/response.js";

class MagasinController {
  async update(req, res, next) {
```

```
try {
  const id = Number.parseInt(req.params.id, 10);

  if (Number.isNaN(id) || id <= 0) {
    return next(httpError(400, "id invalide"));
  }

  const updatedMagasin = await magasinService.updateMagasin(id,
req.body ?? {});

  if (!updatedMagasin) {
    return next(httpError(404, "magasin introuvable"));
  }

  return response(res, 200, updatedMagasin);
} catch (error) {
  return next(error);
}

}

async delete(req, res, next) {
  try {
    const id = Number.parseInt(req.params.id, 10);

    if (Number.isNaN(id) || id <= 0) {
      return next(httpError(400, "id invalide"));
    }

    const deleted = await magasinService.deleteMagasin(id);

    if (!deleted) {
      return next(httpError(404, "magasin introuvable"));
    }

    return res.status(204).send();
  } catch (error) {
    return next(error);
  }
}
```

```
}  
}  
  
async create(req, res, next){  
  try {  
    const newMagasin = await magasinService.createMagasin(req.body ?? {});  
    return response(res, 201, newMagasin);  
  } catch (error) {  
    return next(error);  
  }  
}  
  
//recupérer tous les magasins  
async getAll(req, res, next){  
  try {  
    const magasins = await magasinService.getAllMagasins();  
    return response(res, 200, magasins);  
  } catch (error) {  
    return next(error);  
  }  
}  
  
//recupérer un magasin  
  
async getById(req, res, next){  
  const id = Number.parseInt(req.params.id, 10);  
  if (Number.isNaN(id) || id <= 0) {  
    return next(httpError(400, "id invalide"));  
  }  
  
  const magasin = await magasinService.getMagasinById(id);  
  if (!magasin) {  
    return next(httpError(404, "magasin introuvable"));  
  }  
  
  return response(res, 200, magasin);  
}
```

```
}  
}  
  
export default new MagasinController();
```

09. exemple de repository

```
import prisma from "../config/prisma.js";  
  
class MagasinRepository {  
  findById(id) {  
    return prisma.magasin.findUnique({  
      where: { id },  
    });  
  }  
  
  updateById(id, data) {  
    return prisma.magasin.update({  
      where: { id },  
      data,  
    });  
  }  
  
  deleteById(id) {  
    return prisma.magasin.delete({  
      where: { id },  
    });  
  }  
  
  // creer magasin  
  async create(data) {  
    return prisma.magasin.create({data})  
  }  
  
  // récupérer tous les magasins  
  async findAll(){  
    return prisma.magasin.findMany({
```

```
        orderBy: { id: "asc" }
    });
}

// récupérer un magasin par ID

async findById(id) {
    return prisma.magasin.findUnique({
        where: { id },
    });
}
}

export default new MagasinRepository();
```

12. exemple de routes

```
import express from "express"
import validateMiddleware from "../middlewares/validate.js";
import employerController from "../controllers/employe.controller.js";
import employeSchema from "../validations/employe.schema.js";

const router = express.Router();

router.post("/", validateMiddleware(employeSchema),
employerController.create );
router.get("/", employerController.getAll);
router.get("/deleted", employerController.getDeleted);
router.get("/:id", employerController.getById);
router.put("/:id", validateMiddleware(employeSchema),
employerController.update);
router.delete("/:id", employerController.delete);
router.patch("/:id/restore", employerController.restore);
```

```
export default router;
```

11. Bonnes pratiques

- Placer **routes statiques** avant **routes dynamiques** (`/deleted` avant `/:id`)
- Toujours utiliser **soft delete** (`deletedAt`) au lieu de supprimer définitivement
- Centraliser la logique métier dans `services`
- Utiliser `repositories` pour **accès aux données Prisma**
- Valider toutes les entrées via Zod / middleware

12. Documentation interactive

Après avoir démarré le serveur :

Swagger UI → <http://localhost:3000/api-docs>